

**DETEKSI DAN KLASIFIKASI RAMBU LALU LINTAS
INDONESIA MENGGUNAKAN VISION
TRANSFORMER**

SKRIPSI

Disusun sebagai salah satu syarat untuk memperoleh Gelar Sarjana Teknik pada
Program Studi Teknik Informatika Universitas Sangga Buana YPKP

Disusun Oleh :

BAGAS SETYO NUGROHO

2113221069



**FAKULTAS TEKNIK
UNIVERSITAS SANGGA BUANA YPKP
BANDUNG
2026**

LEMBAR PENGESAHAN

Tugas Akhir ini diajukan oleh :

Nama : Bagas Setyo Nugroho

NPM : 2113221069

Program Studi : Teknik Informatika

Judul : DETEKSI DAN KLASIFIKASI RAMBU LALU LINTAS
INDONESIA MENGGUNAKAN VISION TRANSFORMER

Telah dipertahankan pada Sidang Usulan Tugas Akhir Semester Ganjil Tahun 2026 di hadapan para penguji dan diterima sebagai bagian dari persyaratan yang diperlukan untuk memperoleh gelar Sarjana Teknik (ST) pada Fakultas Teknik Program Studi S1 Teknik Informatika Universitas Sangga Buana.

Bandung, 30 Januari 2026

Pembimbing

Bambang Sugiarto, S.T., M.T.

NIDN. 8861060017

Penguji I

Penguji II

Nama
NIDN.

Nama
NIDN.

Mengetahui
Ketua Program Studi Teknik Informatika

Gunawan, S.T., M.Kom., MOS., MTA., MCE.

NIDN. 040427604

KATA PENGANTAR

Puji syukur saya panjatkan ke hadirat Tuhan Yang Maha Esa, karena berkat rahmat dan karunia-Nya, saya dapat menyusun laporan usulan penelitian dengan judul "***Deteksi dan Klasifikasi Rambu Lalu Lintas Indonesia menggunakan Vision Transformer***". Penelitian ini merupakan salah satu upaya untuk mengembangkan teknologi yang dapat meningkatkan sistem deteksi rambu lalu lintas, yang memiliki peranan penting dalam keamanan dan efisiensi transportasi, terutama dalam sistem bantuan pengemudi (ADAS) dan analitik keselamatan lalu lintas.

Penelitian ini direncanakan untuk mengimplementasikan model *Vision Transformer* (ViT), khususnya YOLOS (*You Only Look at One Sequence*), dalam mendeteksi dan mengklasifikasikan rambu lalu lintas Indonesia, serta membangun prototipe sistem berbasis web yang memungkinkan pengguna untuk mengunggah gambar dan memperoleh hasil deteksi setelah pengguna mengunggah gambar. Rencana pelaksanaan penelitian meliputi beberapa tahapan penting, mulai dari pengumpulan *dataset*, pengembangan model deteksi, hingga pembuatan antarmuka pengguna yang ramah dan fungsional.

Saya menyadari bahwa penelitian ini tidak akan dapat tersusun dengan baik tanpa dukungan, bantuan, dan bimbingan dari berbagai pihak. Oleh karena itu, dengan penuh rasa terima kasih, saya ingin mengucapkan terima kasih yang sebesar-besarnya kepada:

1. Gunawan, S.T., M.Kom., MOS., MTA., MCE. selaku Ketua Program Studi, atas arahan dan dukungan yang diberikan selama proses penyusunan Usulan Tugas Akhir (UTA) ini.
2. Bambang Sugiarto, S.T., M.T. selaku Dosen Pembimbing UTA, atas bimbingan, arahan, serta masukan yang sangat berharga dalam penyusunan penelitian ini.

3. Keluarga dan teman-teman yang selalu memberikan dukungan moral, doa, dan semangat sepanjang perjalanan penelitian ini.
4. Seluruh pihak yang telah berkontribusi baik secara langsung maupun tidak langsung dalam penelitian ini, yang tidak dapat saya sebutkan satu per satu.

Saya berharap hasil dari penelitian ini dapat memberikan kontribusi yang bermanfaat, baik dalam pengembangan teknologi pengemudian cerdas maupun untuk aplikasi-aplikasi lain yang membutuhkan sistem deteksi objek berbasis visi komputer. Selain itu, penelitian ini juga diharapkan dapat membuka peluang untuk penelitian lebih lanjut dalam meningkatkan ketahanan model terhadap tantangan-tantangan visual seperti kondisi pencahayaan yang buruk dan *occlusion* pada objek.

Akhir kata, saya berharap penelitian ini dapat memberikan manfaat bagi pengembangan ilmu pengetahuan dan teknologi, serta bermanfaat bagi masyarakat, khususnya dalam meningkatkan keselamatan berkendara.

Bandung, 30 Januari 2026

Bagas Setyo Nugroho

ABSTRAK

Deteksi dan klasifikasi rambu lalu lintas adalah aspek penting dalam sistem bantuan pengemudi (ADAS) dan analitik keselamatan lalu lintas. Ketepatan dalam mendeteksi dan mengklasifikasikan rambu lalu lintas dapat meningkatkan keselamatan dan efisiensi dalam berkendara. Namun, tantangan utama dalam implementasi sistem ini adalah variasi kondisi visual seperti perubahan pencahayaan, *occlusion*, serta variasi ukuran dan sudut pandang rambu. Penelitian ini mengusulkan penggunaan Vision Transformer (ViT), khususnya model YOLOS (You Only Look at One Sequence), untuk deteksi dan klasifikasi rambu lalu lintas Indonesia. Dengan memanfaatkan kemampuan self-attention yang dimiliki oleh ViT, sistem ini dapat menangkap konteks global dalam citra, berpotensi meningkatkan ketahanan deteksi pada variasi kondisi visual dihadapkan pada kondisi jalan yang beragam. *Dataset* yang digunakan berasal dari Kaggle, yang berisi gambar rambu lalu lintas Indonesia dengan anotasi *bounding box*. Model YOLOS diterapkan untuk mendeteksi berbagai jenis rambu, dan sistem ini juga untuk mengukur kinerja model. Selain itu, penelitian ini juga mengembangkan antarmuka pengguna berbasis web menggunakan FastAPI dan React untuk memudahkan pengguna mengunggah gambar rambu lalu lintas dan memperoleh hasil deteksi secara langsung setelah gambar diunggah. Dengan penerapan sistem berbasis Vision Transformer, penelitian ini bertujuan untuk memberikan solusi deteksi rambu lalu lintas sebagai prototipe yang dapat dikembangkan lebih lanjut untuk implementasi pada skenario nyata.

Kata Kunci: Deteksi Rambu Lalu Lintas, Vision Transformer, YOLOS, Aplikasi, Deteksi Objek, Pengemudian Cerdas.

DAFTAR ISI

LEMBAR PENGESAHAN	i
KATA PENGANTAR.....	ii
ABSTRAK	iv
DAFTAR ISI.....	v
DAFTAR GAMBAR.....	vii
DAFTAR TABEL	viii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang Masalah	1
1.2 Rumusan Masalah	3
1.3 Tujuan Penelitian.....	3
1.4 Batasan Masalah.....	3
1.5 Metodologi Penelitian	4
1.6 Sistematika Penulisan.....	5
BAB II LANDASAN TEORI	6
2.1 Penelitian Terdahulu.....	6
2.2 Rambu Lalu Lintas Indonesia.....	8
2.2 Computer Vision dan Object Detection	11
2.3 Format Anotasi YOLO	11
2.4 Vision Transformer	12
2.5 YOLOS untuk Deteksi Objek.....	13
2.6 Teknologi Pendukung Sistem.....	15
2.6.1 FastAPI	15
2.6.2 React	16
2.6.3 Docker.....	17
2.6.4 SQLite.....	17
BAB III PERANCANGAN.....	19
3.1 Metode Penelitian.....	19
3.1.1 Teknik Pengumpulan Data.....	19
3.1.2 Model Pengembangan Sistem.....	19
3.2 Gambaran Umum Sistem	20
3.3 Data Penelitian	23
3.4 Perancangan Metode	25

3.5 Perancangan Arsitektur Sistem	29
3.6 Perancangan UML.....	30
3.6.1 Use Case Diagram	30
3.6.2 Activity Diagram Proses Training	31
3.6.3 Activity Diagram Proses Prediksi.....	33
3.6.4 Sequence Diagram Prediksi	33
3.7 Perancangan Antarmuka.....	34
3.7.1 Dashboard.....	35
3.7.2 Training & Model	35
3.7.3 Dataset	36
3.7.4 Prediksi	37
DAFTAR PUSTAKA.....	40

DAFTAR GAMBAR

Gambar 3.1 Training Model.....	21
Gambar 3.2 Prediksi Gambar	22
Gambar 3.3 Alur Pipeline Persiapan Data untuk Pelatihan Model YOLOs/ViT .	27
Gambar 3.4 Arsitektur Sistem Deteksi dan Klasifikasi Rambu Lalu Lintas.....	29
Gambar 3.5 Use Case Diagram Prediksi Rambu Lalu Lintas	31
Gambar 3.6 Activity Diagram Proses Training.....	32
Gambar 3.7 Activity Diagram Proses Prediksi	33
Gambar 3.8 Sequence Diagram Prediksi	34
Gambar 3.9 Antarmuka Dashboard.....	35
Gambar 3.10 Antarmuka Training	36
Gambar 3.11 Antarmuka Dataset	37
Gambar 3.12 Antarmuka Prediksi.....	38

DAFTAR TABEL

Tabel 2.1 Perbandingan Penelitian Terdahulu	7
Tabel 3.1 Mapping Kelas	23
Tabel 3.2 Struktur Dataset.....	24
Tabel 3.3 Data Anotasi.....	25

BAB I

PENDAHULUAN

1.1 Latar Belakang Masalah

Rambu lalu lintas merupakan komponen penting dalam sistem transportasi, berfungsi untuk memberikan informasi dan petunjuk kepada pengguna jalan. Fungsi utamanya adalah untuk menjaga keselamatan dan ketertiban berkendara, serta memastikan arus lalu lintas berjalan dengan aman dan efisien [1]. Namun, meskipun keberadaannya sangat vital, tidak semua pengemudi dapat memahami makna atau arti dari setiap jenis rambu lalu lintas, terutama di Indonesia, di mana keragaman rambu, serta perbedaan dalam hal desain, warna, dan ukuran, menjadi tantangan besar. Hal ini ditambah dengan keterbatasan pengemudi dalam memahami arti simbol-simbol pada rambu-rambu lalu lintas, terutama bagi mereka yang kurang berpengalaman atau pengemudi yang sering berkendara di jalan-jalan yang tidak mereka kenal [2]. Oleh karena itu, sistem otomatis yang dapat mendeteksi dan mengklasifikasikan rambu lalu lintas dengan lebih baik menjadi sangat penting untuk meningkatkan keselamatan dan efisiensi berkendara.

Masalah ini semakin kompleks dengan meningkatnya volume kendaraan dan tingginya mobilitas masyarakat, baik untuk keperluan sehari-hari maupun pariwisata. Jalan raya di Indonesia sering kali dipenuhi dengan kendaraan yang padat dan keberagaman kondisi jalan yang menyulitkan pengemudi dalam mendeteksi dan menginterpretasikan rambu-rambu lalu lintas, terutama ketika rambu tersebut terhalang oleh kendaraan besar atau objek lain. Selain itu, faktor lingkungan seperti cuaca, pencahayaan yang buruk, dan kondisi jalan yang tidak terawat sering kali membuat pengemudi kesulitan membaca dan memahami informasi yang terdapat pada rambu-rambu tersebut [3].

Dalam konteks ini, diperlukan solusi teknologi yang dapat membantu meningkatkan pemahaman dan keterbacaan informasi rambu lalu lintas secara otomatis dan akurat [4]. Selain faktor lingkungan, permasalahan rambu lalu lintas di lapangan juga berkaitan dengan kondisi fisik rambu itu sendiri. Di beberapa lokasi, rambu dapat mengalami pudar warna, tergores, tertutup stiker, miring, atau bahkan sebagian tertutup pepohonan dan elemen jalan lainnya. Kondisi tersebut

menyebabkan informasi pada rambu tidak terbaca secara jelas, sehingga meningkatkan risiko salah interpretasi oleh pengemudi, terutama ketika keputusan berkendara harus diambil dalam waktu singkat [2].

Permasalahan juga muncul karena karakteristik rambu lalu lintas yang cenderung berukuran relatif kecil jika ditangkap melalui kamera dari jarak tertentu. Dalam situasi berkendara, rambu sering muncul sebagai objek kecil pada citra, sementara latar belakang jalan sangat bervariasi dan ramai. Hal ini menyulitkan proses identifikasi rambu secara konsisten karena rambu dapat “tenggelam” di antara objek lain dan tidak selalu berada pada posisi yang sama di setiap pengambilan gambar [10].

Tantangan lain adalah adanya perbedaan sudut pandang dan distorsi perspektif. Rambu bisa terlihat berbeda tergantung posisi kamera, jarak, kemiringan, dan arah kendaraan. Perubahan sudut pengambilan ini dapat mengubah proporsi bentuk rambu pada citra, sehingga sistem pengenalan perlu mampu menangani variasi tampilan yang tinggi agar hasil deteksi dan klasifikasi tidak mudah keliru [10].

Di sisi implementasi, kebutuhan sistem yang diharapkan tidak hanya mampu menyatakan “jenis rambu”, tetapi juga harus mampu menunjukkan lokasi rambu pada gambar. Kebutuhan ini penting agar hasil sistem dapat dipahami pengguna, dapat divalidasi dengan mudah, serta berpotensi dikembangkan untuk mendukung kebutuhan lain seperti dokumentasi pelanggaran, pemantauan fasilitas jalan, maupun analisis keselamatan lalu lintas. Karena itu, diperlukan pendekatan pengolahan citra yang dapat bekerja stabil pada berbagai kondisi nyata di jalan, bukan hanya pada kondisi gambar yang ideal [7].

Berdasarkan permasalahan tersebut, penelitian ini memfokuskan pada pengembangan sistem yang mampu mendeteksi sekaligus mengklasifikasikan rambu lalu lintas Indonesia dari citra secara otomatis. Diharapkan sistem yang diusulkan dapat membantu meningkatkan pemahaman rambu bagi pengguna jalan, sekaligus menjadi dasar pengembangan lebih lanjut untuk kebutuhan keselamatan berkendara dan pengelolaan transportasi berbasis data [9].

1.2 Rumusan Masalah

Berdasarkan latar belakang tersebut, maka rumusan masalah dalam penelitian ini adalah:

- a. Bagaimana membangun model yang dapat melakukan deteksi dan klasifikasi rambu lalu lintas Indonesia menggunakan Vision Transformer (ViT) berbasis YOLOs?
- b. Bagaimana merancang sistem berbasis web yang memungkinkan pengguna melakukan unggah gambar dan menampilkan hasil prediksi kelas rambu serta *bounding box* menggunakan FastAPI dan React?
- c. Bagaimana melakukan analisis sederhana terhadap hasil prediksi untuk mengetahui kinerja model dan menyusun rekomendasi pengembangan (misalnya perbaikan data atau pengaturan pelatihan)?

1.3 Tujuan Penelitian

- a. Mengembangkan model yang mampu melakukan deteksi dan klasifikasi rambu lalu lintas Indonesia menggunakan Vision Transformer (ViT) berbasis YOLOs.
- b. Membangun sistem berbasis web menggunakan FastAPI dan React yang dapat menerima unggah gambar serta menampilkan hasil prediksi kelas rambu dan *bounding box*.
- c. Melakukan analisis sederhana terhadap hasil prediksi guna mengetahui kinerja model dan menyusun rekomendasi pengembangan (misalnya perbaikan data atau pengaturan pelatihan).

1.4 Batasan Masalah

Agar pembahasan penelitian tidak terlalu luas dan tetap terfokus pada tujuan yang ingin dicapai, penelitian ini dibatasi pada beberapa ketentuan berikut:

1. Data penelitian menggunakan *dataset* rambu lalu lintas Indonesia yang bersumber dari Kaggle, dengan struktur folder train dan test serta anotasi *bounding box* berformat YOLO (.txt).
2. Input sistem berupa gambar statis yang diunggah pengguna melalui antarmuka web (bukan video maupun streaming).

3. Metode deteksi dan klasifikasi rambu dibatasi menggunakan pendekatan Vision Transformer, yaitu model YOLOS (You Only Look at One Sequence).
4. Keluaran sistem dibatasi pada hasil prediksi berupa *bounding box*, label kelas rambu, dan *confidence score*, serta visualisasi hasil deteksi pada gambar.
5. Penelitian tidak membahas pengenalan teks pada rambu (OCR), pelacakan objek, maupun pengambilan keputusan untuk kendaraan otonom.
6. Sistem dijalankan pada lingkungan lokal menggunakan Docker dengan teknologi Python (*backend* dan model), React (*frontend*), serta SQLite untuk penyimpanan metadata model dan status model aktif.
7. Evaluasi kuantitatif menggunakan metrik deteksi objek (misalnya mAP/IoU) tidak menjadi fokus pada tahap ini; pengujian difokuskan pada uji fungsional alur sistem dan analisis kualitatif terhadap hasil deteksi pada data uji.

1.5 Metodologi Penelitian

Metodologi penelitian ini disusun sebagai rangkaian langkah untuk mencapai tujuan pengembangan prototipe deteksi dan klasifikasi rambu lalu lintas Indonesia berbasis Vision Transformer (YOLOS). Tahapan penelitian meliputi:

1. Studi literatur terkait deteksi objek, rambu lalu lintas, Vision Transformer, dan YOLOS;
2. Akuisisi *dataset* rambu lalu lintas Indonesia dari Kaggle serta pemeriksaan struktur data train dan test;
3. Persiapan data yang mencakup validasi pasangan file gambar–label dan parsing anotasi berformat YOLO untuk membentuk target *bounding box* yang kompatibel dengan *pipeline* pelatihan;
4. Implementasi dan pelatihan model YOLOS menggunakan data latih untuk menghasilkan model deteksi;

5. Penyimpanan model hasil pelatihan dalam versi tertentu serta pencatatan metadata model pada SQLite untuk mendukung pemilihan model aktif;
6. Pengembangan *backend* menggunakan FastAPI untuk layanan training, manajemen model, dan prediksi;
7. Pengembangan *frontend* menggunakan React untuk antarmuka training, pemilihan model aktif, dan unggah gambar;
8. Pengujian fungsional sistem melalui skenario training dan prediksi pada data uji, serta analisis kualitatif hasil deteksi (*bounding box*, label, confidence); dan
9. Menjalankan sistem pada lingkungan lokal berbasis Docker untuk memastikan konsistensi environment dan kemudahan replikasi.

1.6 Sistematika Penulisan

Sistematika penulisan laporan usulan penelitian ini disusun untuk memudahkan pembaca memahami alur dan isi pembahasan. Adapun sistematika penulisan adalah sebagai berikut:

1. Bab I Pendahuluan berisi latar belakang masalah, rumusan masalah, tujuan penelitian, batasan masalah, metodologi penelitian, serta sistematika penulisan.
2. Bab II Landasan Teori memuat penelitian terdahulu dan teori-teori yang berkaitan dengan rambu lalu lintas Indonesia, computer vision dan object detection, format anotasi YOLO, Vision Transformer (ViT), YOLOs untuk deteksi objek, serta teknologi pendukung sistem yang digunakan.
3. Bab III Perancangan menjelaskan metode penelitian dan rancangan sistem yang akan dibangun, meliputi gambaran umum sistem, data penelitian, perancangan metode, perancangan arsitektur sistem, perancangan UML, serta perancangan antarmuka (dashboard, training & model, *dataset*, dan prediksi).

BAB II

LANDASAN TEORI

2.1 Penelitian Terdahulu

Penelitian terdahulu yang menjadi landasan dalam usulan penelitian ini dapat dikelompokkan menjadi tiga bagian, yaitu penelitian yang membahas urgensi dan efektivitas rambu lalu lintas, penelitian berbasis computer vision dan model deteksi berbasis CNN/YOLO sebagai pembanding, serta penelitian yang mengarah pada pemanfaatan Transformer (Vision Transformer) untuk tugas deteksi rambu.

Pada aspek urgensi dan konteks, beberapa penelitian menekankan pentingnya rambu lalu lintas dalam mendukung ketertiban dan keselamatan berkendara serta kebutuhan penertiban pelanggaran lalu lintas. Rohim membahas upaya penertiban pelanggaran lalu lintas yang menunjukkan bahwa kepatuhan terhadap aturan lalu lintas masih menjadi tantangan di lapangan [1]. Harahap dkk. mengevaluasi keefektifan rambu lalu lintas dalam meningkatkan keselamatan jalan raya, sehingga menguatkan bahwa rambu merupakan komponen penting yang perlu dipahami dan dipatuhi pengguna jalan [2]. Selain itu, Supartini dkk. menyoroti penegakan hukum berbasis kamera (ETLE) dan peran pengawasan sebagai upaya meningkatkan kepatuhan berlalu lintas [3], sedangkan Thalib dkk. membahas optimalisasi e-tilang sebagai inovasi digital untuk meningkatkan kesadaran hukum dan ketertiban lalu lintas [4]. Kelompok penelitian ini memperkuat latar belakang bahwa keselamatan dan kepatuhan berlalu lintas masih memerlukan dukungan pendekatan berbasis teknologi.

Pada aspek teknis pengolahan citra, penelitian-penelitian berbasis computer vision menunjukkan bahwa pengenalan objek dapat dilakukan melalui pendekatan deteksi dan klasifikasi. Agma mengulas penerapan computer vision untuk deteksi objek sebagai dasar konsep pemrosesan citra dan interpretasi hasil deteksi [10]. Dalam konteks rambu lalu lintas, Al’Fattah dkk. membandingkan model CNN untuk klasifikasi rambu lalu lintas di Indonesia, yang menunjukkan bahwa pendekatan CNN dapat digunakan sebagai baseline klasifikasi namun masih berada pada keluarga CNN [6]. Pangestu & Putra menerapkan pendekatan YOLOv9 untuk deteksi rambu lalu lintas pada kondisi pencahayaan yang

bervariasi, sehingga menegaskan adanya tantangan kondisi nyata (misalnya pencahayaan) pada tugas deteksi [9]. Liyanthy juga membahas rancangan data *pipeline* untuk pelatihan agen otonom dalam mengenali rambu lalu lintas, yang relevan sebagai gambaran kebutuhan data dan proses pelatihan agar model siap digunakan dalam sistem [7]. Kelompok penelitian ini menegaskan bahwa tugas rambu tidak hanya klasifikasi, namun juga memerlukan deteksi (lokasi/*bounding box*) dengan mempertimbangkan variasi kondisi citra.

Selanjutnya, pada tren metode terbaru, Transformer (Vision Transformer) semakin banyak diteliti untuk meningkatkan kemampuan model pada tugas pengenalan/deteksi. Fawole & Rawat menyajikan survei terkait ketahanan Vision Transformer terhadap serangan *adversarial* pada deteksi dan pengenalan rambu lalu lintas, yang menunjukkan bahwa Transformer merupakan arah penelitian yang relevan pada domain rambu lalu lintas [5]. Pramesti & Rochadiani menerapkan kombinasi CNN dan Transformer untuk image *captioning* pada citra rambu lalu lintas Indonesia, yang menunjukkan pemanfaatan Transformer pada konteks rambu Indonesia, meskipun tugasnya berbeda karena berfokus pada deskripsi/*captioning*, bukan deteksi *bounding box* [8]. Berdasarkan penelitian-penelitian tersebut, masih terdapat ruang untuk mengusulkan pendekatan berbasis Vision Transformer yang secara spesifik diarahkan untuk deteksi dan klasifikasi rambu pada rambu lalu lintas Indonesia, serta diintegrasikan ke dalam prototipe sistem berbasis web untuk kebutuhan pengguna. Ringkasan perbandingan penelitian terdahulu disajikan pada Tabel 2.1.

Tabel 2.1 Perbandingan Penelitian Terdahulu

No	Penelitian	Metode	Keluaran	Kaitan terhadap penelitian ini
2	Deteksi rambu pada variasi pencahayaan [9]	YOLOv9	Hasil deteksi berupa lokasi objek (<i>bounding box</i>) dan kelas rambu, disertai nilai keyakinan (<i>confidence score</i>)	Menjadi pembanding deteksi berbasis CNN; penelitian ini mengusulkan pendekatan Vision

				Transformer (YOLOS/ViT)
3	Klasifikasi rambu Indonesia [6]	CNN (Xception, InceptionV3)	Hasil klasifikasi berupa label/kelas rambu	Berfokus pada klasifikasi tanpa lokasi; penelitian ini menargetkan deteksi sekaligus klasifikasi (lokasi + kelas)
4	Image <i>captioning</i> rambu Indonesia [8]	CNN + Transformer	Hasil berupa keterangan teks (<i>caption</i>) yang merepresentasikan makna rambu	Tugas berbeda (pembuatan <i>caption</i>); penelitian ini berfokus pada deteksi dan klasifikasi

2.2 Rambu Lalu Lintas Indonesia

Rambu lalu lintas merupakan salah satu komponen penting dalam penyelenggaraan lalu lintas karena berfungsi sebagai media komunikasi visual yang memberikan informasi, peringatan, larangan, perintah, maupun petunjuk kepada pengguna jalan. Di Indonesia, rambu lalu lintas memiliki landasan pengaturan yang jelas melalui regulasi teknis. Peraturan Menteri Perhubungan Republik Indonesia Nomor PM 13 Tahun 2014 mendefinisikan rambu lalu lintas sebagai bagian perlengkapan jalan yang dapat berupa lambang, huruf, angka, kalimat, dan/atau perpaduannya, yang berfungsi sebagai peringatan, larangan, perintah, atau petunjuk bagi pengguna jalan. Definisi ini menegaskan bahwa rambu bukan sekadar elemen dekoratif, melainkan instrumen keselamatan dan ketertiban lalu lintas yang wajib dipahami oleh pengendara maupun pejalan kaki.

Secara umum, rambu lalu lintas dibutuhkan untuk mengurangi ambiguitas dalam pengambilan keputusan di jalan, terutama pada kondisi lalu lintas yang padat, perubahan kondisi jalan yang cepat, atau ketika pengemudi menghadapi lingkungan yang tidak familiar. Dalam konteks keselamatan, kepatuhan terhadap rambu

berperan dalam menurunkan potensi kecelakaan akibat kesalahan manusia, misalnya pengemudi tidak menyadari adanya larangan, tidak membaca perintah jalur, atau terlambat merespons peringatan bahaya. Selain itu, rambu juga mendukung efisiensi arus lalu lintas karena mengarahkan pengendara mengambil tindakan yang tepat, misalnya memilih lajur tertentu, mengurangi kecepatan, atau mengikuti aturan prioritas.

Berdasarkan regulasi PM 13 Tahun 2014, rambu lalu lintas dikelompokkan ke dalam empat jenis utama, yaitu rambu peringatan, rambu larangan, rambu perintah, dan rambu petunjuk. Pengelompokan ini penting karena setiap jenis rambu memiliki karakteristik dan tujuan komunikasi yang berbeda. Rambu peringatan bertujuan memberi informasi mengenai potensi bahaya atau kondisi khusus di depan, sehingga pengemudi dapat bersiap (misalnya tikungan tajam, jalan menurun, atau persimpangan tertentu). Rambu larangan menandai tindakan yang tidak boleh dilakukan, seperti dilarang berhenti, dilarang parkir, atau dilarang memutar balik. Rambu perintah mengarahkan tindakan yang harus dilakukan oleh pengendara, misalnya wajib belok kiri atau wajib mengikuti jalur tertentu. Sementara itu, rambu petunjuk memberikan informasi arah, lokasi, fasilitas, atau informasi layanan yang membantu navigasi pengguna jalan.

Selain pembagian jenis, standar rambu juga terkait dengan aspek teknis seperti spesifikasi, penyelenggaraan, dan pembuatan rambu. PM 13 Tahun 2014 menyebut ruang lingkup pengaturan mencakup spesifikasi teknis rambu, penyelenggaraan rambu, dan pembuatan rambu. Dari sisi teknis, rambu dapat berbentuk rambu konvensional maupun rambu elektronik, dan rambu konvensional umumnya menggunakan material yang memiliki kemampuan memantulkan cahaya (*retroreflektif*) untuk menjaga keterbacaan pada malam hari atau kondisi cahaya rendah. Keberadaan standar ini membuat desain rambu cenderung konsisten, sehingga pengguna jalan dapat mengenali pola warna, bentuk, dan piktogram secara lebih cepat.

Walaupun rambu telah distandarkan, pada praktiknya kondisi visual rambu di lapangan dapat sangat bervariasi. Tantangan umum yang muncul mencakup perubahan pencahayaan (*siang–malam*, bayangan, lampu kendaraan), cuaca (*hujan/kabut*), *occlusion* (rambu tertutup kendaraan/pohon), degradasi fisik (*pudar*,

kotor, rusak), serta variasi sudut pandang dan ukuran rambu di dalam citra. Kondisi-kondisi ini membuat pengenalan rambu secara otomatis menjadi lebih kompleks [5]. Hal tersebut juga terlihat pada penelitian deteksi rambu, misalnya studi YOLOv9 untuk deteksi rambu di bawah variasi pencahayaan yang menekankan pentingnya ketahanan model terhadap kondisi pencahayaan [9].

Dalam konteks penelitian berbasis visi komputer, karakteristik rambu lalu lintas yang cenderung memiliki bentuk geometris dan simbol tertentu menjadikannya objek yang relevan untuk tugas deteksi objek (*object detection*). Deteksi objek bertujuan menemukan lokasi (*bounding box*) sekaligus mengklasifikasikan jenis objek yang terdeteksi. Rambu lalu lintas sering kali berukuran relatif kecil pada citra jalan, dan dapat muncul lebih dari satu rambu dalam satu gambar, sehingga dibutuhkan model yang mampu menangani multi-object detection sekaligus tetap sensitif pada objek kecil. Kondisi ini selaras dengan data anotasi *bounding box*, termasuk format anotasi yang menyimpan informasi kelas dan koordinat *bounding box* seperti pada *dataset* berformat YOLO yang umum dipakai dalam eksperimen deteksi.

Selain deteksi rambu sebagai objek, terdapat pendekatan lain yang menekankan pemahaman semantik rambu, misalnya menghasilkan deskripsi (*caption*) dari gambar rambu untuk membantu pemahaman pengguna [8]. Penelitian *image captioning* rambu Indonesia yang memanfaatkan CNN terlatih dan Transformer menunjukkan bahwa rambu dapat diproses lebih lanjut untuk menghasilkan informasi tekstual yang mudah dipahami, walaupun fokusnya berbeda dengan deteksi objek. Dengan demikian, penelitian rambu lalu lintas dapat berkembang dari sekadar mengenali “jenis rambu” menjadi memahami “makna rambu” dan bagaimana informasi tersebut disajikan.

Berdasarkan pembahasan di atas, rambu lalu lintas Indonesia memiliki standar definisi dan klasifikasi yang jelas, namun implementasi pengenalan otomatis di kondisi nyata menghadapi tantangan visual yang beragam. Karena itulah penelitian deteksi dan klasifikasi rambu tetap relevan untuk dikembangkan, khususnya dengan pendekatan model modern yang diharapkan mampu menangkap konteks visual secara lebih baik, serta diintegrasikan ke dalam prototipe sistem yang dapat digunakan untuk uji coba prediksi berbasis unggah gambar.

2.2 Computer Vision dan Object Detection

Computer vision merupakan bidang yang mempelajari bagaimana komputer dapat memahami informasi visual dari citra atau video [10]. Dalam praktiknya, tugas computer vision beragam, mulai dari klasifikasi citra, lokalisasi, hingga deteksi objek yang menggabungkan keduanya: sistem mendeteksi satu atau lebih objek di dalam gambar sekaligus menentukan kelasnya.

Object detection berfokus pada dua keluaran utama, yaitu lokasi objek dan kategori objek. Lokasi biasanya direpresentasikan menggunakan *bounding box* (kotak pembatas) yang menggambarkan posisi objek pada citra, sedangkan kategori berupa label kelas beserta nilai keyakinan (*confidence score*) [10]. Representasi *bounding box* dapat dinyatakan dalam beberapa format, salah satunya menggunakan titik tengah (x_center , y_center) serta lebar dan tinggi kotak (*width*, *height*), yang juga umum dipakai pada anotasi berformat YOLO [9].

Dalam pengembangan sistem deteksi, data beranotasi menjadi komponen penting. *Dataset* biasanya berisi gambar dan anotasi ground truth berupa *bounding box* serta label kelas untuk setiap objek. Model kemudian dilatih untuk mempelajari pola visual objek dan menghasilkan prediksi *bounding box* serta label kelas pada data baru. Secara umum, pendekatan deteksi modern dapat dikelompokkan menjadi *two-stage detector* dan *one-stage detector*.

Pada penelitian ini, konsep object detection digunakan untuk mendeteksi dan mengklasifikasikan rambu lalu lintas pada gambar statis. Sistem menerima gambar yang diunggah pengguna, kemudian model menghasilkan keluaran berupa lokasi rambu (*bounding box*) serta jenis rambu (label kelas). Dengan pendekatan ini, satu gambar dapat memuat lebih dari satu rambu dan sistem tetap dapat memberikan hasil deteksi untuk setiap rambu yang teridentifikasi.

2.3 Format Anotasi YOLO

Pada penelitian deteksi objek, anotasi berfungsi sebagai ground truth yang menjelaskan kelas objek dan lokasi objek pada sebuah gambar. Salah satu format anotasi yang umum digunakan adalah format YOLO, yaitu satu file teks (.txt) untuk setiap gambar dengan satu baris anotasi untuk setiap objek yang muncul pada

gambar tersebut [11]. Setiap baris berisi lima nilai: `class_id` `x_center` `y_center` `width` `height`. Nilai `class_id` merepresentasikan indeks kelas, sedangkan empat nilai berikutnya merepresentasikan *bounding box* dalam format center-based.

Koordinat *bounding box* pada format YOLO biasanya menggunakan nilai ternormalisasi dalam rentang 0 sampai 1. Artinya, `x_center` dan `width` dinormalisasi terhadap lebar gambar, sedangkan `y_center` dan `height` dinormalisasi terhadap tinggi gambar. Normalisasi ini memudahkan penggunaan *dataset* pada berbagai ukuran resolusi karena anotasi tidak bergantung pada satuan piksel tertentu. Jika anotasi awal berbasis piksel, konversi dilakukan dengan membagi nilai koordinat terhadap dimensi gambar.

Dalam implementasi *dataset* YOLO, satu gambar dapat memiliki lebih dari satu objek sehingga file anotasinya dapat berisi lebih dari satu baris. Sebaliknya, apabila sebuah gambar tidak mengandung objek, dapat terjadi file anotasi tidak disediakan. Karena penelitian ini menggunakan citra rambu lalu lintas yang pada beberapa kasus dapat memuat lebih dari satu rambu dalam satu gambar, struktur multi-baris pada anotasi YOLO menjadi relevan untuk mendukung skenario tersebut.

2.4 Vision Transformer

Vision Transformer (ViT) merupakan pendekatan untuk memproses citra menggunakan arsitektur transformer yang sebelumnya populer pada pemrosesan bahasa. Konsep utama ViT adalah mengubah citra menjadi sekumpulan patch berukuran tetap, kemudian setiap patch direpresentasikan sebagai token sehingga citra dapat dipandang sebagai urutan token [12]. Token-token ini kemudian diproses oleh transformer *encoder* yang menggunakan mekanisme self-attention untuk memodelkan hubungan antar patch secara global. Dengan demikian, ViT memiliki kemampuan menangkap konteks menyeluruh dari citra, tidak hanya fitur lokal pada area tertentu.

Secara ringkas, alur ViT meliputi: pembagian citra menjadi patch, pembentukan embedding patch, penambahan informasi posisi, lalu pemrosesan melalui beberapa lapisan transformer *encoder*. Pendekatan ini membuat ViT bergantung pada representasi global antar bagian citra, sehingga banyak penelitian

menggunakan ViT untuk berbagai tugas visi komputer, termasuk klasifikasi dan deteksi objek. Dalam penelitian ini, ViT menjadi dasar bagi penggunaan model YOLOs yang memanfaatkan mekanisme transformer untuk menghasilkan prediksi lokasi dan kelas objek pada citra rambu.

2.5 YOLOs untuk Deteksi Objek

YOLOS (You Only Look at One Sequence) merupakan model deteksi objek berbasis Vision Transformer (ViT) yang dirancang dengan prinsip utama meminimalkan modifikasi terhadap arsitektur ViT standar [13]. Tujuan dari YOLOs adalah menunjukkan bahwa transformer, jika dipandang sebagai pemroses sequence token, dapat digunakan untuk tugas deteksi objek tanpa banyak inductive bias khusus deteksi seperti yang lazim pada detektor konvensional. Pada dasarnya, YOLOs mengubah masalah deteksi objek menjadi proses sequence-to-sequence yang sederhana: citra diubah menjadi urutan token, lalu model menghasilkan prediksi objek dalam bentuk sekumpulan keluaran.

Secara konsep, YOLOs berangkat dari ViT yang memecah citra menjadi patch dan mengubah setiap patch menjadi patch embedding. Berbeda dari ViT untuk klasifikasi yang umumnya memiliki satu token khusus seperti [CLS], YOLOs mengganti token klasifikasi tersebut dengan sejumlah token deteksi yang berperan sebagai “slot” prediksi objek. Setiap token deteksi ini kemudian, setelah diproses oleh lapisan transformer *encoder*, akan digunakan untuk menghasilkan keluaran berupa *bounding box* dan kelas objek. Dengan cara ini, YOLOs dapat memprediksi beberapa objek sekaligus, dan setiap token deteksi diarahkan untuk “mewakili” satu objek yang terdeteksi.

Dalam proses pelatihan, YOLOs menggunakan pendekatan set-based prediction yang sejalan dengan ide deteksi end-to-end, yaitu model menghasilkan sekumpulan prediksi objek dan dipasangkan dengan anotasi ground truth menggunakan mekanisme pencocokan pada fungsi loss. Pada dokumentasi model YOLOs di Hugging Face juga dijelaskan bahwa YOLOs adalah ViT yang dilatih dengan loss seperti pada DETR, sehingga keluaran prediksi dapat diperlakukan sebagai satu set deteksi tanpa bergantung pada anchor tradisional. Konsep ini membantu mengurangi kebutuhan komponen deteksi tambahan yang kompleks,

karena arsitektur inti tetap berupa transformer *encoder* yang memproses token secara global.

YOLOS tersedia dalam beberapa varian ukuran model yang membedakan kapasitas parameter dan kebutuhan komputasi. Pada penelitian ini, pemanfaatan YOLOS relevan karena model mampu menghasilkan keluaran deteksi berupa lokasi dan label kelas yang sesuai dengan format anotasi data rambu lalu lintas. Implementasi praktisnya dapat dilakukan melalui ekosistem Hugging Face Transformers, misalnya menggunakan *YolosForObjectDetection* dan komponen pemrosesan citra yang menyiapkan input sesuai kebutuhan model. Ini mempermudah integrasi YOLOS ke dalam *pipeline* pelatihan dan inferensi berbasis Python, serta mendukung pengembangan prototipe sistem (training dan prediksi) melalui *backend*.

Dari sisi alur kerja, penggunaan YOLOS untuk deteksi rambu lalu lintas dapat dipandang dalam tiga tahap sederhana: (1) persiapan input, (2) inferensi model, dan (3) interpretasi output. Pada tahap persiapan input, gambar diubah menjadi representasi yang sesuai, lalu dibentuk menjadi token patch [12]. Pada tahap inferensi, transformer *encoder* memproses urutan token patch bersama token deteksi untuk menghasilkan representasi akhir token deteksi. Pada tahap interpretasi output, setiap token deteksi menghasilkan prediksi kotak pembatas dan kelas, yang kemudian dapat ditampilkan sebagai *bounding box* pada gambar beserta label dan *confidence score*. Mekanisme ini selaras dengan kebutuhan sistem penelitian, yaitu menerima gambar statis dari pengguna dan mengeluarkan hasil deteksi yang dapat divisualisasikan [13].

Walaupun YOLOS menawarkan desain yang sederhana, terdapat beberapa hal yang perlu diperhatikan saat mengadaptasikannya pada data rambu lalu lintas. Pertama, kualitas anotasi *bounding box* dan konsistensi label kelas sangat berpengaruh karena model dilatih secara end-to-end dari data beranotasi. Kedua, karena rambu pada citra jalan sering berukuran kecil dan bervariasi kondisi pencahayaan, hasil deteksi sangat bergantung pada variasi data latih dan pengaturan pelatihan. Dengan memahami karakteristik YOLOS sebagai detektor berbasis transformer yang memanfaatkan konteks global citra, penelitian ini memanfaatkan YOLOS sebagai pendekatan utama untuk membangun prototipe deteksi dan

klasifikasi rambu lalu lintas Indonesia pada citra yang diunggah pengguna melalui sistem berbasis web.

2.6 Teknologi Pendukung Sistem

2.6.1 FastAPI

FastAPI merupakan framework Python untuk membangun layanan web yang dirancang ringan dan cepat, serta mendukung pembuatan *endpoint* berbasis HTTP untuk kebutuhan aplikasi modern [14]. Pada pengembangan aplikasi berbasis layanan, API berperan sebagai penghubung antar komponen sistem—misalnya antara antarmuka pengguna (*frontend*) dan layanan pemrosesan (*backend*). FastAPI menyediakan pola pengembangan API yang terstruktur, mulai dari pendefinisian *endpoint*, metode HTTP (GET/POST), hingga pengiriman dan penerimaan data dalam format terstandar seperti JSON. Selain itu, FastAPI memanfaatkan type hints Python untuk membantu validasi data masukan/keluaran secara lebih konsisten, sehingga pengembang dapat mengurangi kesalahan format data ketika sistem saling bertukar informasi [14].

Keunggulan lain dari FastAPI adalah dukungannya terhadap standar dokumentasi API, seperti OpenAPI, yang memungkinkan spesifikasi *endpoint*, parameter, dan respons terdokumentasi dengan jelas. Dokumentasi yang baik membantu proses pengujian dan integrasi, karena alur komunikasi antara *frontend* dan *backend* menjadi lebih mudah dipahami dan divalidasi. Pada konteks penelitian, aspek ini penting agar alur sistem yang diusulkan dapat didemonstrasikan secara jelas: mulai dari pengguna mengunggah gambar, sistem memproses permintaan, hingga sistem mengembalikan hasil prediksi yang dapat divisualisasikan pada antarmuka [14].

Pada penelitian ini, FastAPI digunakan sebagai *backend* untuk menjembatani proses training, manajemen model, dan prediksi [14]. *Backend* menerima input berupa gambar statis yang diunggah pengguna, melakukan prapemrosesan sesuai kebutuhan model YOLO/ ViT, menjalankan inferensi menggunakan model aktif, lalu mengembalikan hasil berupa *bounding box*, label kelas, dan *confidence score* dalam format terstruktur. Dengan pendekatan ini, proses komputasi model tetap berada di sisi server/*backend*, sedangkan *frontend*

berfokus pada interaksi pengguna dan visualisasi hasil. Pemisahan peran ini mendukung prinsip arsitektur sistem yang lebih jelas: *backend* menangani logika pemrosesan dan komputasi, sementara *frontend* menangani penyajian informasi kepada pengguna.

Selain untuk prediksi, penggunaan *backend* API juga relevan untuk skenario pengelolaan model. Karena penelitian ini memungkinkan adanya lebih dari satu model, *backend* dapat bertindak sebagai pusat kontrol untuk memilih model aktif, menyimpan metadata model, dan memastikan bahwa proses prediksi menggunakan model yang tepat. Dengan demikian, FastAPI tidak hanya berfungsi sebagai “penerima file dan pengirim hasil”, tetapi juga sebagai komponen yang menjaga integrasi proses penelitian mulai dari pengelolaan versi model hingga penyajian hasil deteksi agar dapat berjalan secara terstruktur dan dapat diuji secara sistematis [14].

2.6.2 React

React merupakan library JavaScript untuk membangun antarmuka pengguna (UI) dengan pendekatan berbasis komponen (component-based architecture). Pada pendekatan ini, tampilan aplikasi dibagi menjadi bagian-bagian kecil (komponen) yang memiliki tanggung jawab spesifik, misalnya komponen unggah gambar, komponen pratinjau, komponen hasil prediksi, dan komponen daftar model. Pemecahan UI menjadi komponen membantu menjaga konsistensi tampilan, mempermudah pemeliharaan, serta memungkinkan pengembangan fitur dilakukan secara bertahap tanpa mengganggu bagian lain [15].

Dalam konteks aplikasi berbasis web, React juga menerapkan konsep pembaruan tampilan secara reaktif melalui mekanisme state dan props. Ketika data berubah, React dapat memperbarui bagian tampilan yang relevan tanpa perlu memuat ulang halaman. Hal ini penting untuk aplikasi deteksi rambu karena pengalaman pengguna akan lebih baik jika proses unggah, pemrosesan, dan penampilan hasil terjadi secara responsif.

Selain itu, React umum digunakan untuk membangun *Single Page Application* (SPA), yaitu aplikasi web yang berpindah antar halaman tanpa reload penuh [15]. Pola SPA sesuai untuk prototipe penelitian karena halaman training, pemilihan model, dan prediksi dapat disusun sebagai modul-modul UI yang

terintegrasi. Di sisi integrasi sistem, React berperan sebagai lapisan presentasi yang berkomunikasi dengan *backend* melalui request API untuk mengirim data dan menerima respons hasil deteksi yang kemudian divisualisasikan pada antarmuka.

2.6.3 Docker

Docker merupakan platform containerisasi yang memungkinkan aplikasi dijalankan dalam lingkungan terisolasi (container) dengan konfigurasi yang konsisten. Container mengemas aplikasi beserta dependensi pentingnya (*library, runtime, konfigurasi*) sehingga aplikasi dapat dijalankan dengan perilaku yang relatif sama pada perangkat yang berbeda. Konsep ini penting karena pengembangan sistem berbasis machine learning sering melibatkan banyak dependensi, yang rawan menimbulkan perbedaan hasil ketika dijalankan pada lingkungan yang tidak seragam [16].

Secara konsep, container berbeda dengan *virtual machine* (VM). VM memvirtualisasikan sistem operasi lengkap, sedangkan container memanfaatkan kernel sistem operasi host dan hanya mengisolasi proses serta dependensi aplikasi [16]. Akibatnya, container umumnya lebih ringan dan cepat dijalankan, sehingga sesuai untuk kebutuhan prototipe dan pengujian. Konsistensi environment ini membantu memastikan bahwa proses pelatihan model, inferensi, serta layanan *backend–frontend* dapat direplikasi tanpa hambatan konfigurasi.

Dalam penelitian ini, Docker mendukung penyusunan sistem sebagai beberapa layanan agar dapat dijalankan secara terkoordinasi. Dengan penggunaan container, proses instalasi lebih terstandar, pengaturan dependensi lebih terkendali, serta risiko “berjalan di laptop saya tapi gagal di laptop lain” dapat dikurangi. Hal ini mendukung tujuan penelitian yang tidak hanya menghasilkan model, tetapi juga prototipe sistem yang dapat diuji dan didemonstrasikan secara stabil.

2.6.4 SQLite

SQLite adalah sistem manajemen basis data relasional yang bersifat embedded dan ringan, dengan penyimpanan dalam satu file database. Berbeda dengan DBMS server, SQLite dapat digunakan langsung oleh aplikasi tanpa instalasi server database [17]. Karakteristik ini membuat SQLite cocok untuk kebutuhan prototipe penelitian karena proses konfigurasi lebih sederhana, mudah

dipindahkan, dan cukup untuk menyimpan data terstruktur dalam skala kecil hingga menengah [17].

Secara konsep, SQLite mendukung prinsip relasional seperti tabel, relasi, query SQL, indeks, dan transaksi. Meski ringan, SQLite tetap memadai untuk kebutuhan pencatatan metadata yang konsisten. Dalam penelitian ini, SQLite digunakan untuk menyimpan metadata model, seperti versi model, lokasi penyimpanan file model, waktu pelatihan, serta penanda model yang sedang aktif. Pencatatan metadata ini penting karena sistem yang mendukung pelatihan berulang akan menghasilkan banyak versi model. Tanpa mekanisme pencatatan, pemilihan model yang digunakan untuk prediksi akan rawan tertukar atau sulit dilacak ketika terjadi kesalahan.

Dengan adanya database metadata, sistem dapat melakukan pergantian model secara terkontrol. Selain itu, penyimpanan ini membantu aspek audit sederhana: sistem dapat mengetahui model mana yang dipakai saat menghasilkan suatu hasil prediksi. Dengan demikian, SQLite berfungsi sebagai komponen pendukung yang menjaga keteraturan pengelolaan versi model dalam prototipe tanpa memerlukan database server terpisah.

BAB III

PERANCANGAN

3.1 Metode Penelitian

3.1.1 Teknik Pengumpulan Data

Teknik pengumpulan data pada penelitian ini menggunakan tiga pendekatan utama yang bertujuan untuk memperoleh data yang akurat dan relevan guna mengembangkan model deteksi dan klasifikasi rambu lalu lintas Indonesia menggunakan *Vision Transformer* (ViT).

1. Akuisisi Dataset Publik

Dataset dikumpulkan melalui platform Kaggle yang menyediakan *dataset* rambu lalu lintas Indonesia dalam format citra digital dan anotasi. Proses akuisisi dilakukan dengan mengunduh *dataset*, memeriksa lisensi penggunaan, serta memvalidasi struktur data yang mencakup jumlah kelas, jumlah gambar, dan format anotasi *bounding box*. *Dataset* publik dipilih karena telah memiliki label terstandarisasi sehingga dapat digunakan langsung untuk proses pelatihan dan evaluasi model deteksi objek.

2. Kurasi dan Validasi Data

Setelah *dataset* diperoleh, dilakukan kurasi untuk memastikan kualitas data dan kesesuaian anotasi. Tahap ini meliputi pengecekan konsistensi label kelas, validitas koordinat *bounding box*, keberadaan file yang rusak/duplikat, serta keseragaman format anotasi. Data yang tidak memenuhi kriteria kualitas dipisahkan agar tidak mengganggu proses pelatihan.

3.1.2 Model Pengembangan Sistem

Model pengembangan sistem ini terdiri dari dua bagian utama, yaitu pengembangan model (YOLOS/ViT) dan pengembangan perangkat lunak (*backend–frontend*), yang dirancang untuk sebagai prototipe sistem deteksi dan klasifikasi rambu lalu lintas berbasis unggah gambar.

1. Persiapan Data & Konversi Anotasi

Dataset pada penelitian ini diperoleh dari Kaggle dan disusun dalam struktur folder train dan test, di mana setiap kelas rambu ditempatkan pada

folder tersendiri. Setiap file gambar berformat .jpg memiliki pasangan file anotasi .txt dengan format YOLO, yaitu `class_id x_center y_center width height` dalam nilai ter-normalisasi. Satu file anotasi dapat memuat satu atau lebih baris anotasi apabila dalam satu gambar terdapat lebih dari satu rambu. Pada tahap persiapan data dilakukan pemeriksaan kesesuaian pasangan gambar-label, validasi format anotasi, serta proses pembacaan (parsing) anotasi untuk membentuk struktur target *bounding box* yang kompatibel dengan *pipeline* pelatihan model YOLO/ ViT.

2. Pengembangan Model

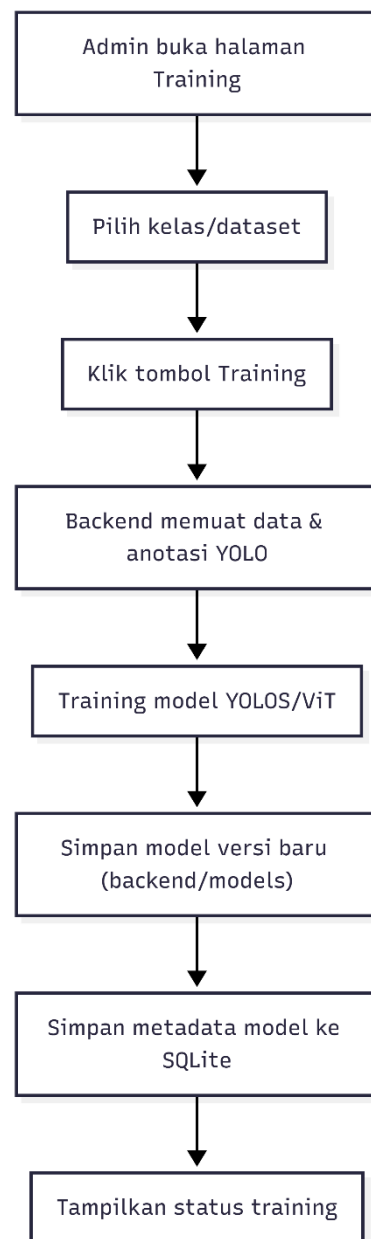
Pengembangan model dilakukan menggunakan pendekatan Vision Transformer dengan model YOLO/ ViT untuk tugas deteksi dan klasifikasi rambu lalu lintas. Proses pelatihan memanfaatkan data pada subset train untuk menghasilkan model yang mampu memprediksi lokasi objek (*bounding box*) beserta label kelas dan *confidence score*. Model hasil pelatihan kemudian disimpan pada direktori `backend/models` dengan penamaan berbasis versi (misalnya `models-v1`, `models-v2`) agar proses eksperimen dan pergantian model dapat dilakukan dengan lebih terkontrol. Informasi model seperti versi, lokasi penyimpanan, dan status model aktif dicatat pada SQLite sehingga sistem prediksi selalu menggunakan model yang sedang ditetapkan sebagai model aktif.

3.2 Gambaran Umum Sistem

Sistem yang dirancang pada penelitian ini merupakan prototipe aplikasi berbasis web untuk mendukung proses training model dan prediksi deteksi rambu lalu lintas Indonesia menggunakan pendekatan Vision Transformer, yaitu YOLO/ ViT. Sistem dijalankan pada lingkungan lokal menggunakan Docker agar dependensi dan konfigurasi lingkungan tetap konsisten. Secara umum, sistem terdiri dari tiga komponen utama, yaitu *frontend* (React) sebagai antarmuka pengguna, *backend* (FastAPI) sebagai layanan pemrosesan dan penghubung ke model, serta SQLite sebagai penyimpanan metadata model dan status model aktif.

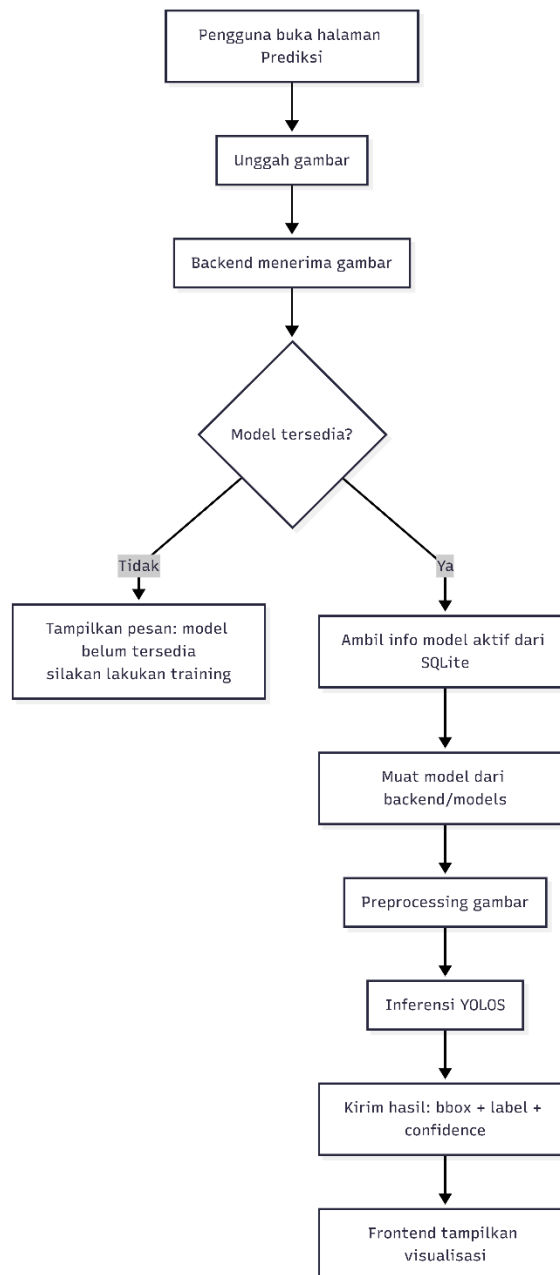
Alur kerja sistem dibagi menjadi dua proses utama. Pertama adalah proses training, di mana admin mengakses halaman training untuk menjalankan pelatihan

model berdasarkan *dataset* yang tersedia. *Backend* akan memuat data latih, membaca anotasi *bounding box* berformat YOLO, dan melakukan pelatihan model YOLOs sesuai konfigurasi yang ditentukan. Setelah training selesai, model disimpan ke folder backend/models dalam bentuk versi tertentu agar hasil eksperimen dapat dikelola dan digunakan kembali. Metadata mengenai versi model, lokasi penyimpanan, dan status model aktif kemudian dicatat pada SQLite. Alur proses training model pada sisi admin ditunjukkan pada Gambar 3.1.



Gambar 3.1 Training Model

Proses kedua adalah prediksi, di mana pengguna mengunggah gambar statis melalui halaman prediksi. *Backend* akan memuat model yang sedang aktif berdasarkan informasi pada SQLite, melakukan prapemrosesan gambar, menjalankan inferensi menggunakan YOLOs, lalu mengembalikan keluaran prediksi berupa *bounding box*, label kelas, dan *confidence score*. *Frontend* menampilkan hasil tersebut dalam bentuk visualisasi sehingga pengguna dapat melihat hasil deteksi secara langsung setelah gambar diunggah. Alur proses prediksi rambu pada sisi pengguna ditunjukkan pada Gambar 3.2.



Gambar 3.2 Prediksi Gambar

3.3 Data Penelitian

Data yang digunakan dalam penelitian ini berupa *dataset* rambu lalu lintas Indonesia yang diperoleh dari Kaggle [18]. *Dataset* disusun ke dalam dua subset, yaitu train dan test, dengan struktur direktori berbasis kelas. Setiap kelas rambu ditempatkan pada folder tersendiri dan berisi pasangan file gambar (.jpg) serta anotasi (.txt). Struktur ini dipilih karena sesuai untuk kebutuhan deteksi objek, yaitu menyediakan informasi kelas dan lokasi objek dalam citra. Contoh isi file anotasi ditunjukkan pada Tabel 3.3.

Setiap file anotasi menggunakan format YOLO, di mana satu baris anotasi merepresentasikan satu objek rambu pada gambar. Satu file .txt dapat berisi satu atau lebih baris apabila pada gambar terdapat lebih dari satu rambu. Format setiap baris anotasi adalah `class_id x_center y_center width height`, dengan `class_id` sebagai indeks kelas dan empat nilai lainnya sebagai koordinat *bounding box* dalam format titik tengah dan ukuran kotak dengan nilai ternormalisasi. Dengan format ini, *dataset* dapat digunakan untuk melatih model YOLOs agar menghasilkan prediksi berupa *bounding box* dan label kelas.

Secara implementasi, *dataset* berada pada direktori `data/traffic_sign` yang di dalamnya terdapat folder train dan test. Masing-masing subset memiliki daftar folder kelas yang sama, dan setiap file gambar memiliki file label dengan nama yang berpasangan. Ringkasan pemetaan kelas dan struktur *dataset* ditunjukkan pada Tabel 3.1 dan Tabel 3.2.

Tabel 3.1 Mapping Kelas

Class id	Nama Kelas	Direktori
0	Larangan Berhenti	larangan-berhenti
1	Larangan Masuk Bagi Kendaraan Bermotor dan Tidak Bermotor	larangan-masuk-bagi-kendaraan-bermotor-dan-tidak-bermotor
2	Larangan Parkir	larangan-parkir
3	Lampu Hijau	lampu-hijau
4	Lampu Kuning	lampu-kuning
5	Lampu Merah	lampu-merah

6	Larangan Belok Kanan	larangan-belok-kanan
7	Larangan Belok Kiri	larangan-belok-kiri
8	Larangan Berjalan Terus Wajib Berhenti Sesaat	larangan-berjalan-terus-wajib-berhenti-sesaat
9	Larangan Memutar Balik	larangan-memutar-balik
10	Peringatan Alat Pemberi Isyarat Lalu Lintas	peringatan-alat-pemberi-isyarat-lalu-lintas
11	Peringatan Banyak Pejalan Kaki Menggunakan Zebra Cross	peringatan-banyak-pejalan-kaki-menggunakan-zebra-cross
12	Peringatan Pintu Perlindungan Kereta Api	peringatan-pintu-perlintasan-kereta-api
13	Peringatan Simpang Tiga Sisi Kiri	peringatan-simpang-tiga-sisi-kiri
14	Peringatan Penegasan Rambu Tambahan	peringatan-penegasan-rambu-tambahan
15	Perintah Masuk Jalur Kiri	perintah-masuk-jalur-kiri
16	Perintah Pilihan Memasuki Salah Satu Jalur	perintah-pilihan-memasuki-salah-satu-jalur
17	Petunjuk Area Parkir	petunjuk-area-parkir
18	Petunjuk Lokasi Pemberhentian Bus	petunjuk-lokasi-pemberhentian-bus
19	Petunjuk Lokasi Putar Balik	petunjuk-lokasi-putar-balik
20	Petunjuk Penyeberangan Pejalan Kaki	petunjuk-penyeberangan-pejalan-kaki

Tabel 3.2 Struktur *Dataset*

Subset	Lokasi	Isi Utama
train	data/traffic_sign/train/<nama_kelas>/	.jpg, .txt
test	data/traffic_sign/test/<nama_kelas>/	.jpg, .txt

Tabel 3.3 Data Anotasi

Nama Kelas	File anotasi	Baris anotasi YOLO
Larangan Belok Kanan	larangan belok kanan (1).txt	6 0.514844 0.354911 0.182812 0.321429
Larangan Memutar Balik	larangan memutar balik (1).txt	9 0.464401 0.492933 0.404531 0.484099
Larangan Belok Kiri	larangan belok kiri (1).txt	7 0.301429 0.388041 0.074286 0.139949
Larangan Parkir	larangan parkir (1).txt	2 0.517595 0.335069 0.131965 0.156250
Larangan Masuk Bagi Kendaraan Bermotor dan Tidak Bermotor	larangan masuk bagi kendaraan bermotor dan tidak bermotor (1).txt	1 0.419935 0.417939 0.369281 0.522901

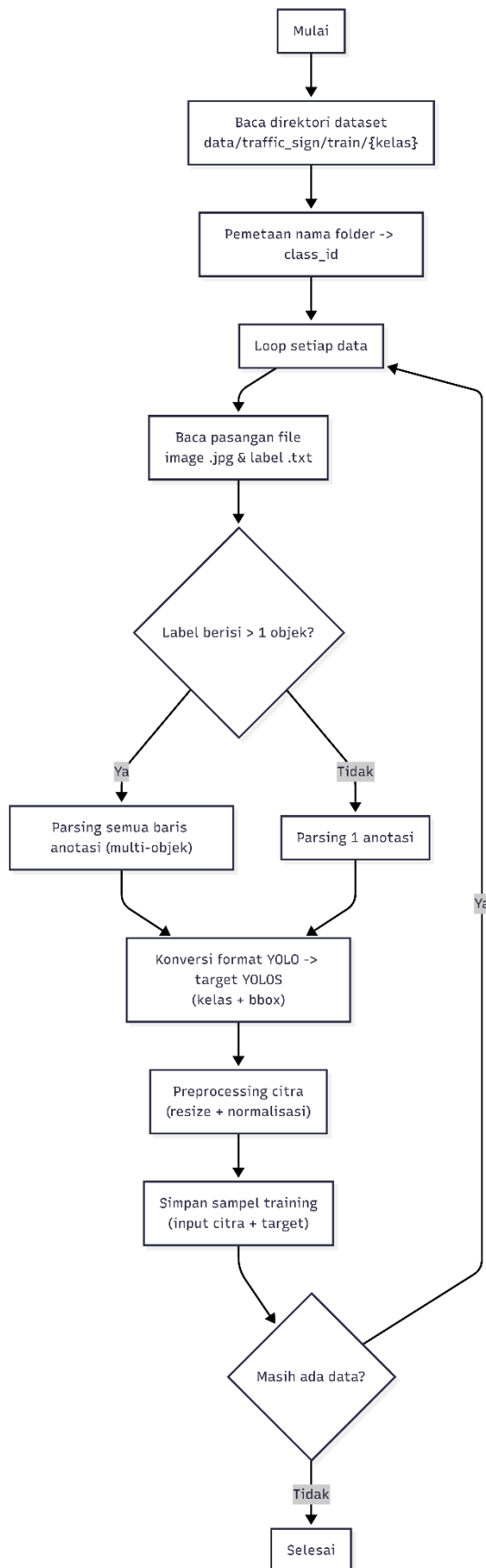
3.4 Perancangan Metode

Perancangan metode pada penelitian ini disusun untuk mengimplementasikan proses deteksi dan klasifikasi rambu lalu lintas berbasis Vision Transformer menggunakan model YOLOS. Metode dirancang sebagai sebuah *pipeline* yang mencakup tahapan persiapan data, pembentukan target anotasi, pelatihan model, penyimpanan model hasil pelatihan, serta penggunaan model untuk proses prediksi pada gambar statis. Perancangan ini selaras dengan struktur *dataset* yang tersedia (train dan test) serta kebutuhan sistem prototipe yang menyediakan fitur training dan prediksi berbasis web.

Tahap awal *pipeline* adalah pembacaan *dataset* dari direktori data/traffic_sign dengan struktur folder per kelas. Pada tahap ini, sistem memetakan nama folder kelas menjadi identitas kelas (*class_id*) yang konsisten, kemudian membaca setiap pasangan file gambar (.jpg) dan file label (.txt). File label berformat YOLO berisi satu atau lebih baris anotasi yang merepresentasikan objek rambu pada gambar. Setiap baris anotasi dibaca untuk memperoleh informasi *class_id* serta koordinat *bounding box* dalam format titik tengah dan ukuran kotak. Karena

beberapa gambar dapat memuat lebih dari satu rambu, *pipeline* dirancang untuk memproses multi-objek dalam satu gambar.

Setelah anotasi dibaca, dilakukan tahap konversi anotasi dan pembentukan target pelatihan agar sesuai dengan kebutuhan input YOLOs. Tahap ini mencakup parsing nilai koordinat YOLO dan membentuk struktur data target *bounding box* yang dipakai pada proses training. Dengan demikian, setiap sampel data pada *dataset* akan menghasilkan dua komponen utama, yaitu input citra dan target label (kelas dan lokasi objek). Pada saat yang sama, citra dipersiapkan melalui proses prapemrosesan, seperti penyesuaian ukuran dan normalisasi, agar sesuai dengan konfigurasi input model. prapemrosesan ini penting untuk menjaga konsistensi bentuk data yang masuk ke model YOLOs pada saat training maupun prediksi. Alur *pipeline* persiapan data dan pembentukan target pelatihan untuk YOLOs/ViT ditunjukkan pada Gambar 3.3.



Gambar 3.3 Alur Pipeline Persiapan Data untuk Pelatihan Model YOLOs/ViT

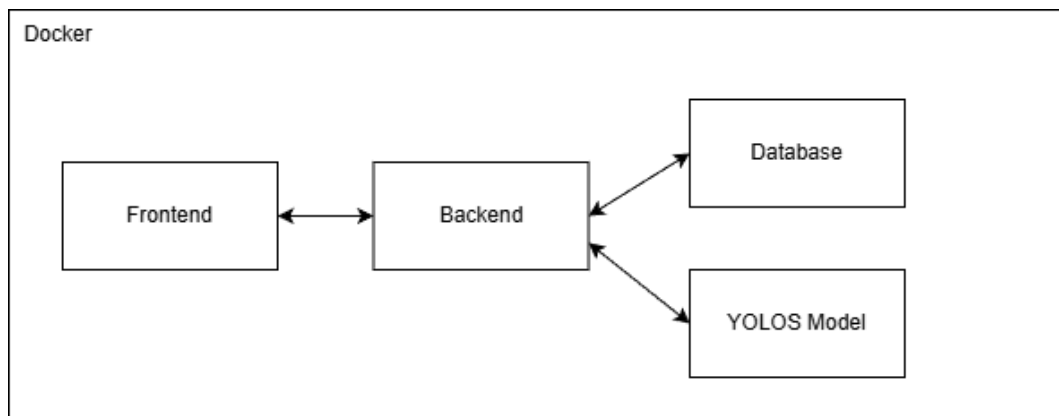
Pada tahapan pelatihan model, YOLOS digunakan sebagai model deteksi objek yang menghasilkan keluaran berupa prediksi *bounding box* dan klasifikasi kelas untuk setiap objek yang terdeteksi. Proses pelatihan dilakukan pada subset train, dengan konfigurasi pelatihan disesuaikan terhadap kebutuhan eksperimen, seperti ukuran input gambar, jumlah epoch, serta parameter optimisasi. Selama training berlangsung, model mempelajari representasi visual rambu lalu lintas dari data latih dan menyesuaikan bobot parameter agar mampu menghasilkan prediksi yang sesuai dengan anotasi. Setelah training selesai, model disimpan ke direktori *backend/models* menggunakan penamaan berbasis versi untuk memudahkan pengelolaan hasil eksperimen.

Agar sistem mendukung pergantian model, dirancang mekanisme manajemen model yang mencatat informasi model dalam SQLite. Informasi yang disimpan mencakup versi model, lokasi penyimpanan, dan status model aktif. Dengan mekanisme ini, sistem dapat memilih model yang akan digunakan pada proses prediksi tanpa perlu melakukan perubahan manual pada kode. Pemilihan model aktif menjadi penting ketika terdapat beberapa model hasil pelatihan dan diperlukan penggunaan model tertentu yang dianggap paling sesuai untuk kebutuhan prediksi.

Tahap terakhir pada *pipeline* adalah prediksi menggunakan model aktif. Pada proses ini, pengguna mengunggah gambar statis melalui antarmuka web, kemudian *backend* memuat model yang ditetapkan sebagai model aktif dari direktori penyimpanan. Gambar di-preprocess dengan langkah yang konsisten dengan training, lalu dilakukan inferensi menggunakan YOLOS untuk menghasilkan keluaran berupa *bounding box*, label kelas, dan *confidence score*. Hasil tersebut kemudian dikembalikan ke *frontend* untuk divisualisasikan, sehingga pengguna dapat melihat posisi rambu yang terdeteksi beserta jenis rambu yang dikenali oleh sistem. Dengan *pipeline* ini, metode yang dirancang mampu mendukung kebutuhan penelitian dalam membangun prototipe deteksi dan klasifikasi rambu lalu lintas berbasis Vision Transformer berbasis unggah gambar.

3.5 Perancangan Arsitektur Sistem

Arsitektur sistem pada penelitian ini dirancang sebagai prototipe berbasis web yang memisahkan tanggung jawab antara antarmuka pengguna, layanan pemrosesan, dan penyimpanan metadata model. Pemisahan ini bertujuan agar proses training dan prediksi model YOLOS/ViT dapat dijalankan secara terstruktur, mudah dikembangkan, serta konsisten ketika dijalankan pada lingkungan lokal. Secara umum, arsitektur terdiri dari empat komponen utama, yaitu *Frontend* (React), *Backend* (FastAPI + Model YOLOS/ViT), Database (SQLite), dan Docker sebagai lingkungan eksekusi. Arsitektur Sistem Deteksi dan Klasifikasi Rambu Lalu Lintas yang diusulkan ditunjukkan pada Gambar 3.4.



Gambar 3.4 Arsitektur Sistem Deteksi dan Klasifikasi Rambu Lalu Lintas

Komponen *frontend* dibangun menggunakan React dan berperan sebagai antarmuka pengguna untuk menjalankan fitur utama sistem. Antarmuka menyediakan halaman untuk melakukan training, memilih model aktif, serta melakukan prediksi melalui unggah gambar. *Frontend* berkomunikasi dengan *backend* melalui request HTTP untuk mengirim data dan menerima respons hasil pemrosesan dari *backend*. Dengan desain berbasis web, pengguna cukup berinteraksi melalui browser tanpa perlu menjalankan proses model secara manual.

Komponen *backend* dikembangkan menggunakan FastAPI sebagai layanan yang menangani proses inti penelitian, yaitu pelatihan dan inferensi model YOLOS/ViT. Pada fitur training, *backend* memuat *dataset* dari struktur folder yang tersedia, memproses anotasi YOLO, menjalankan *pipeline* pelatihan, dan menyimpan model hasil training. Pada fitur prediksi, *backend* menerima gambar

yang diunggah pengguna, melakukan prapemrosesan, memuat model yang sedang aktif, menjalankan inferensi, lalu mengembalikan hasil deteksi berupa *bounding box*, label kelas, dan *confidence score*. *Backend* juga menyediakan layanan manajemen model yang menampilkan daftar model versi dan mendukung penetapan model aktif yang akan digunakan untuk prediksi.

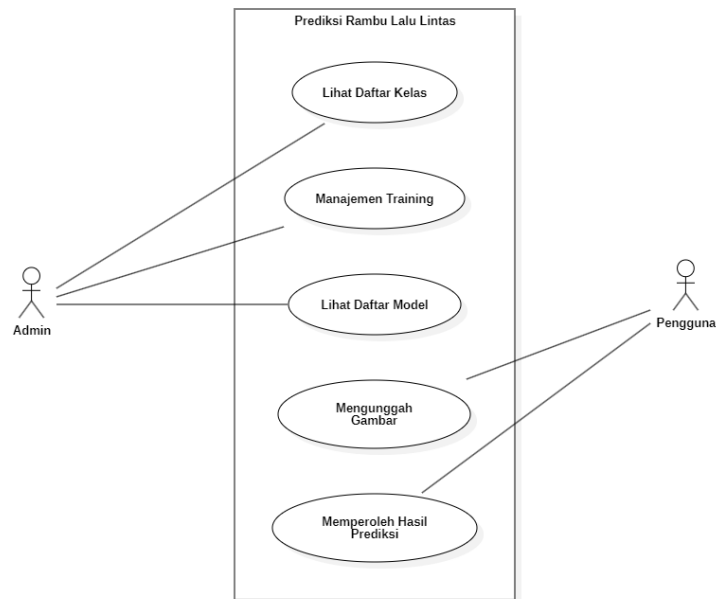
Untuk mendukung pergantian model dan pencatatan hasil eksperimen, sistem menggunakan SQLite sebagai penyimpanan metadata model. Database ini menyimpan informasi seperti nama/versi model, lokasi folder penyimpanan model pada direktori backend/models, serta penanda model mana yang sedang aktif. Dengan penyimpanan metadata tersebut, proses pemilihan model tidak bergantung pada perubahan kode, dan sistem dapat mempertahankan riwayat versi model hasil training. Pemilihan model aktif dari SQLite menjadi acuan *backend* saat melakukan prediksi, sehingga hasil inferensi selalu menggunakan model yang ditetapkan.

Seluruh komponen dijalankan pada lingkungan lokal menggunakan Docker untuk memastikan konsistensi konfigurasi dan dependensi. Docker membantu mengelola dependensi Python serta dependensi *frontend* React dalam lingkungan yang terisolasi. Dengan arsitektur ini, sistem dapat dijalankan dan direplikasi dengan lebih mudah pada perangkat lain, sekaligus mengurangi risiko perbedaan environment yang dapat memengaruhi proses training maupun prediksi. Selain itu, penggunaan Docker memudahkan pengembangan bertahap karena setiap komponen dapat dikelola sebagai layanan yang terpisah namun tetap terintegrasi melalui jaringan container.

3.6 Perancangan UML

3.6.1 Use Case Diagram

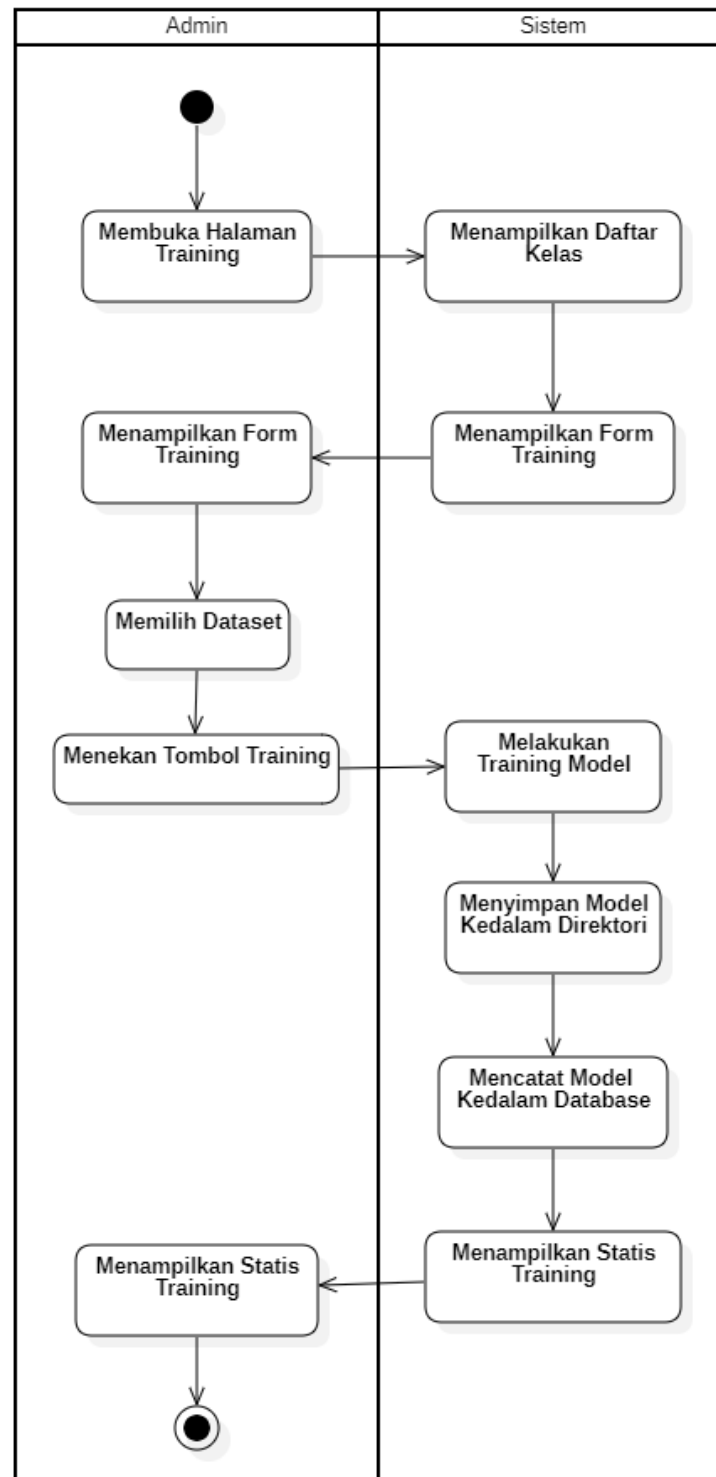
Use case pada sistem berfokus pada dua proses utama: training model dan prediksi. Admin memiliki use case untuk melihat daftar kelas yang tersedia, menjalankan training, melihat daftar model hasil training, serta memilih model aktif. Pengguna memiliki use case untuk mengunggah gambar dan memperoleh hasil prediksi. Hasil prediksi yang ditampilkan berupa *bounding box*, label kelas, dan *confidence score*. Use case diagram pada sistem ditunjukkan pada Gambar 3.5.



Gambar 3.5 Use Case Diagram Prediksi Rambu Lalu Lintas

3.6.2 Activity Diagram Proses Training

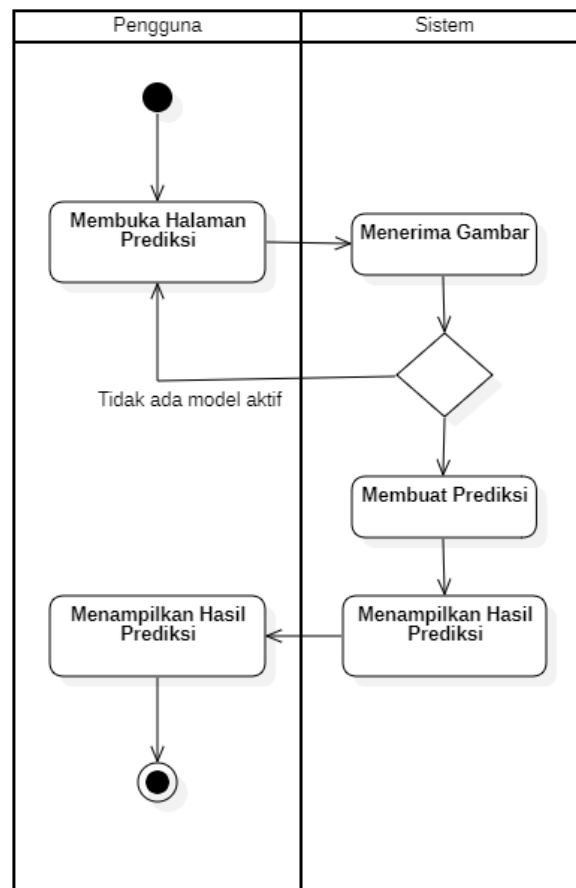
Activity diagram proses training menggambarkan langkah-langkah sistem ketika Admin menjalankan pelatihan model. Alur dimulai dari Admin membuka halaman training, kemudian sistem menampilkan daftar kelas yang tersedia. Admin memilih *dataset* atau kelas yang akan digunakan, lalu menekan tombol training. *Backend* memuat data dan anotasi, menjalankan proses pelatihan YOLO/ ViT, kemudian menyimpan model hasil training ke direktori backend/models sebagai versi baru. Setelah penyimpanan selesai, sistem mencatat metadata model (versi dan path) ke SQLite, lalu menampilkan status training kepada Admin. Alur proses training ditunjukkan pada Gambar 3.6.



Gambar 3.6 Activity Diagram Proses Training

3.6.3 Activity Diagram Proses Prediksi

Activity diagram proses prediksi menjelaskan alur ketika pengguna melakukan deteksi rambu lalu lintas pada gambar statis. Alur dimulai dari pengguna membuka halaman prediksi dan mengunggah gambar. *Backend* menerima gambar, memuat model yang sedang aktif berdasarkan informasi pada SQLite, melakukan prapemrosesan, lalu menjalankan inferensi dengan YOLOs untuk menghasilkan keluaran *bouding box*, label kelas, dan *confidence score*. Hasil prediksi kemudian dikirim kembali ke *frontend* dan divisualisasikan pada gambar yang diunggah. Alur proses prediksi ditunjukkan pada Gambar 3.7.

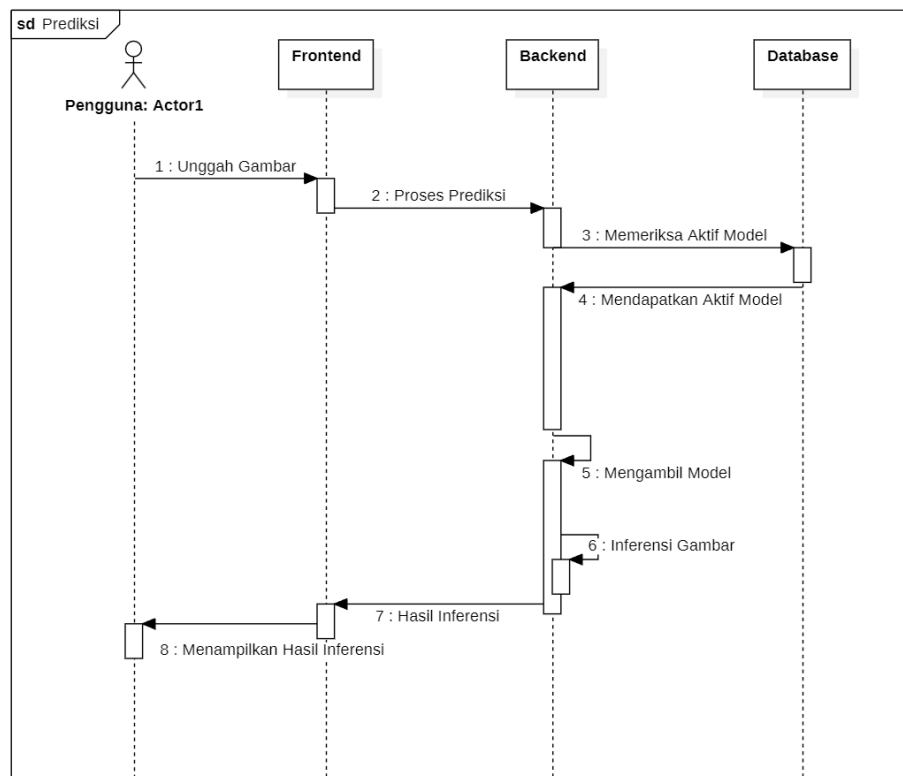


Gambar 3.7 Activity Diagram Proses Prediksi

3.6.4 Sequence Diagram Prediksi

Sequence diagram prediksi menggambarkan interaksi antar komponen sistem saat proses prediksi berlangsung. Urutan komunikasi dimulai dari pengguna melakukan unggah gambar pada *frontend*, kemudian *frontend* mengirim request ke *backend*. *Backend* memeriksa model aktif pada SQLite, memuat model dari

direktori backend/models, melakukan prapemrosesan, menjalankan inferensi, lalu mengembalikan respons hasil prediksi ke *frontend*. *Frontend* menampilkan hasil deteksi dalam bentuk visualisasi *bounding box* beserta label dan *confidence score*. Diagram ini menegaskan pembagian peran antar komponen, di mana *frontend* menangani interaksi dan tampilan, sedangkan *backend* menangani proses model dan pengambilan model aktif. Interaksi antar komponen pada proses prediksi ditunjukkan pada Gambar 3.8.



Gambar 3.8 Sequence Diagram Prediksi

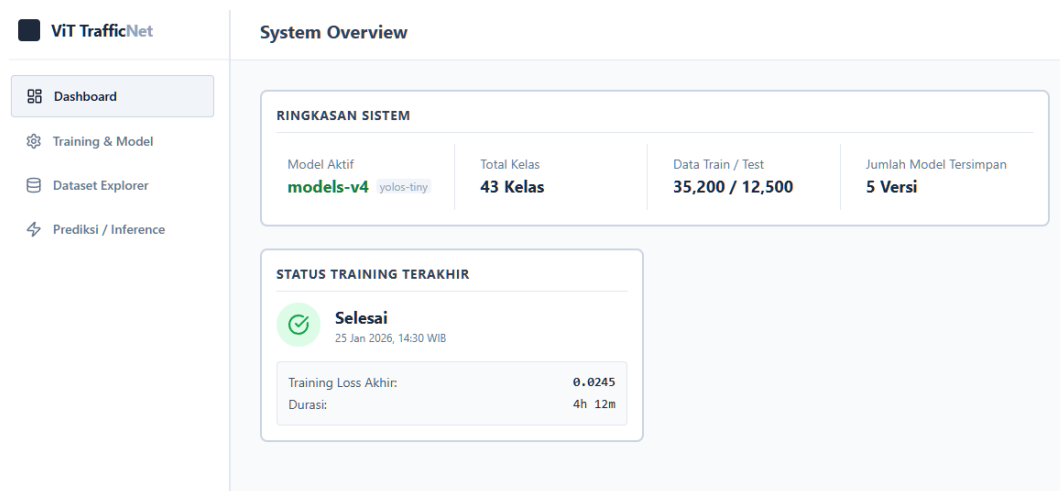
3.7 Perancangan Antarmuka

Perancangan antarmuka pada penelitian ini bertujuan untuk menggambarkan tampilan dan alur interaksi pengguna dalam menjalankan fitur utama sistem. Antarmuka dibangun berbasis web agar mudah digunakan melalui browser, serta memfokuskan fungsi pada kebutuhan prototipe, yaitu pengelolaan *dataset*, proses training model, manajemen model, dan prediksi gambar statis. Struktur menu pada sistem terdiri dari empat bagian utama, yaitu Dashboard,

Training & Model, *Dataset*, dan Prediksi. Rancangan antarmuka untuk setiap menu utama ditunjukkan pada Gambar 3.9 sampai dengan Gambar 3.12.

3.7.1 Dashboard

Menu Dashboard berfungsi sebagai halaman ringkasan untuk menampilkan informasi umum mengenai kondisi sistem. Informasi yang ditampilkan dapat mencakup status model yang sedang aktif, jumlah model yang tersimpan, ringkasan *dataset* yang tersedia, serta ringkasan aktivitas terakhir seperti training terakhir atau model versi terbaru yang dibuat. Dashboard dirancang untuk membantu pengguna memahami kondisi sistem secara cepat sebelum menjalankan proses training atau prediksi. Rancangan tampilan menu Dashboard ditunjukkan pada Gambar 3.9.

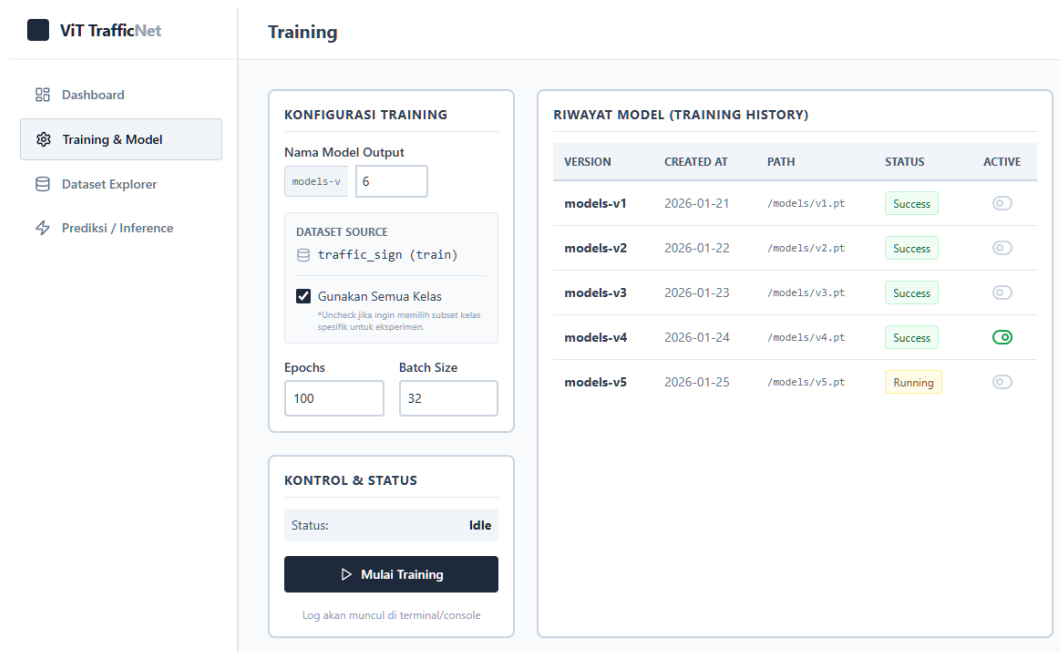


Gambar 3.9 Antarmuka Dashboard

3.7.2 Training & Model

Menu Training & Model menyediakan fitur untuk menjalankan proses pelatihan model sekaligus mengelola model yang telah dihasilkan. Pada halaman ini, sistem menampilkan daftar opsi training, tombol untuk memulai training, serta indikator status proses. Setelah training selesai, halaman ini juga menampilkan daftar versi model yang tersimpan pada direktori backend/models, termasuk informasi dasar seperti nama/versi model dan waktu pembuatan. Selain itu, tersedia kontrol untuk menetapkan model aktif yang akan digunakan pada proses prediksi, sehingga pengguna dapat melakukan pergantian model sesuai kebutuhan.

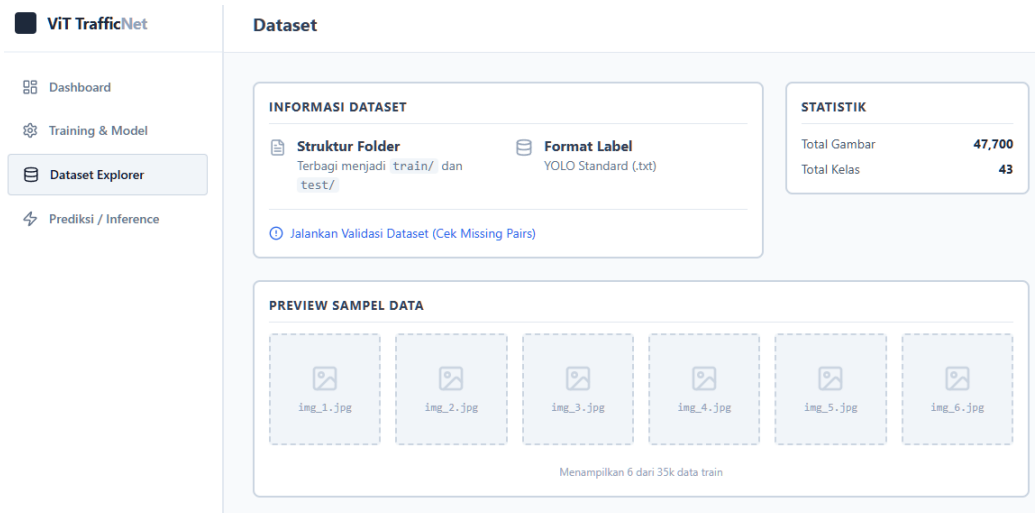
eksperimen. Rancangan tampilan menu Training & Model ditunjukkan pada Gambar 3.10.



Gambar 3.10 Antarmuka Training

3.7.3 Dataset

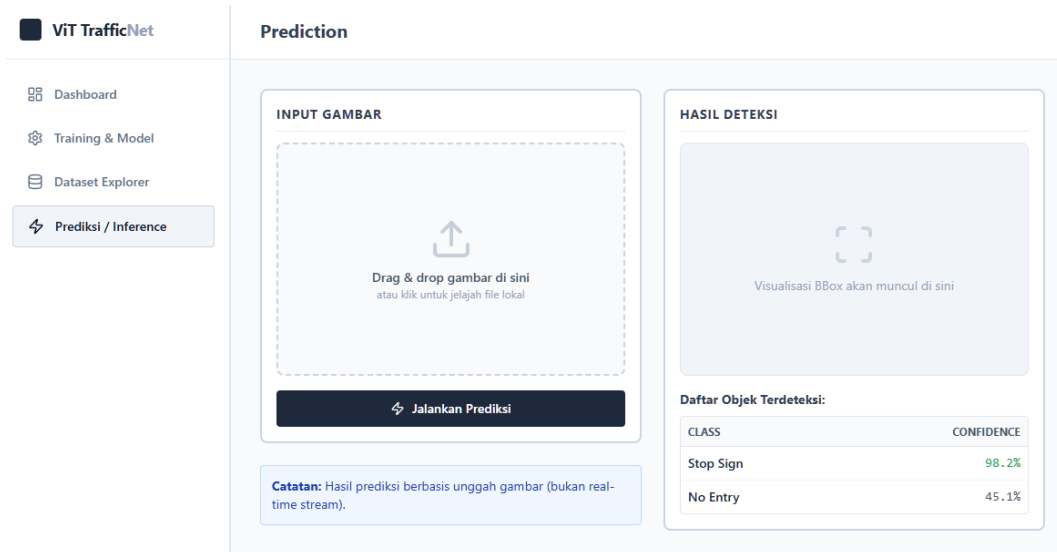
Menu *Dataset* digunakan untuk menampilkan dan memeriksa struktur *dataset* yang digunakan pada penelitian. Halaman ini menampilkan daftar kelas rambu yang tersedia serta informasi ringkas mengenai isi *dataset*, misalnya jumlah gambar per kelas atau struktur folder train dan test. Tujuan menu ini adalah memudahkan pengguna memastikan *dataset* telah tersusun dengan benar, sehingga proses training dapat dijalankan tanpa kendala akibat kesalahan struktur data. Menu *Dataset* juga dapat berfungsi sebagai sarana validasi awal sebelum training dilakukan. Rancangan tampilan menu *Dataset* ditunjukkan pada Gambar 3.11.



Gambar 3.11 Antarmuka Dataset

3.7.4 Prediksi

Menu Prediksi merupakan halaman utama untuk melakukan deteksi dan klasifikasi rambu lalu lintas pada gambar statis. Pada halaman ini, pengguna mengunggah gambar melalui komponen upload, kemudian sistem mengirim gambar ke *backend* untuk diproses menggunakan model yang sedang aktif. Hasil prediksi ditampilkan dalam bentuk visualisasi, yaitu gambar yang diberi *bounding box*, label kelas rambu, dan *confidence score*. Halaman ini dirancang sederhana agar pengguna dapat melakukan prediksi berulang kali dengan gambar yang berbeda, serta memastikan hasil deteksi dapat diamati secara langsung setelah proses unggah selesai. Rancangan tampilan menu Prediksi ditunjukkan pada Gambar 3.12.



Gambar 3.12 Antarmuka Prediksi

KESIMPULAN

Kesimpulan pada usulan penelitian ini menegaskan bahwa deteksi dan klasifikasi rambu lalu lintas Indonesia masih menghadapi tantangan kondisi visual seperti variasi pencahayaan, oklusi, dan perbedaan ukuran objek pada citra. Oleh karena itu, penelitian ini mengusulkan penggunaan pendekatan Vision Transformer, yaitu YOLOS, untuk membangun prototipe sistem deteksi rambu berbasis unggah gambar yang menghasilkan keluaran berupa *bounding box*, label kelas, dan *confidence score*. Sistem dirancang terintegrasi melalui *backend* FastAPI dan *frontend* React serta dijalankan pada lingkungan lokal berbasis Docker, dengan SQLite untuk pengelolaan metadata model dan model aktif. Diharapkan hasil penelitian ini dapat menjadi dasar pengembangan lebih lanjut untuk penerapan pengenalan rambu pada berbagai kebutuhan sistem transportasi cerdas.

DAFTAR PUSTAKA

- [1] D. D. N. Rohim, "Upaya Satlantas Polres Kutai Timur Dalam Menertibkan Pelanggaran Lalu Lintas Di Wilayah Hukum Polres Kutai Timur," *J. Law (Jurnal Ilmu Hukum)*, vol. 5, no. 2, p. 795–809, 2020.
- [2] R. Harahap, P. P. A. Simanjuntak, R. T. Siagian, S. Z. Daily, V. G. Lubis, Y. I. Simbolon and Y. Y. Nababan, "Evaluasi Keefektifan Rambu Lalu Lintas Dalam Meningkatkan Keselamatan Jalan Raya," *Bersatu J. Pendidik. Bhinneka Tunggal Ika*, vol. 1, no. 6, p. 351–356, 2023.
- [3] E. B. Supartini and T. Setiawan, Meningkatkan Kepatuhan dan Kesadaran Diri Berlalu Lintas Melalui Penegakan Hukum Berbasis Kamera (ETLE) dan Pengawasan Polisi, CV Eureka Media Aksara, 2024.
- [4] M. R. Thalib, A. A. K. Ismail and D. E. Moha, "Optimalisasi E-Tilang Dalam Inovasi Digital Untuk Meningkatkan Kesadaran Hukum Dan Ketertiban Lalu Lintas," *SINERGI J. Ris. Ilm.*, vol. 2, no. 1, p. 173–190, 2025.
- [5] O. Fawole and D. Rawat, "Recent Advances in Vision Transformer Robustness Against Adversarial Attacks in Traffic Sign Detection and Recognition: A Survey," *ACM Computing Surveys*, vol. 57, no. 10, pp. 1-269, 2025.
- [6] G. Al’Fattah, B. Purnama and B. Kosala, "Analisis Perbandingan CNN Model Xception Dan InceptionV3 Dalam Pengklasifikasian Rambu Lalu Lintas Di Indonesia," *eProceedings Engineering*, vol. 12, no. 3, 2025.
- [7] M. Liyanthy, "Perancangan Data Pipeline Computer Vision untuk Mendukung Pelatihan Agen Otonom dalam Mengenali Rambu Lalu Lintas," *J. Comput. Inf. Syst. Ampera*, vol. 5, no. 1, p. 45–60, 2024.
- [8] T. H. Pramesti and N. A. Rochadiani, "Image Captioning for Indonesian Traffic Sign Images Using Pretrained CNN and Transformer," *Indonesian Journal of Computer Science*, vol. 13, no. 3, p. 45–52, 2024.
- [9] M. P. K. Pangestu and A. Putra, "YOLOv9-based Traffic Sign Detection under Varying Lighting Conditions," *JUTIF J.*, vol. 6, no. 1, p. 101–110, 2025.

- [10] A. R. Agha, "Penerapan Computer Vision untuk Deteksi Objek dan Pengenalan Wajah," *J. Komput. dan Tek. Inform.*, vol. 1, no. 1, p. 31–37, 2025.
- [11] Ultralytics, "Object Detection Datasets Overview," 21 January 2026. [Online]. Available: <https://docs.ultralytics.com/datasets/detect>.
- [12] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit and N. Houlsby, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," arXiv, 2020.
- [13] Y. Fang, B. Liao, X. Wang, J. Fang, J. Qi, R. Wu, J. Niu and W. Liu, "You Only Look at One Sequence: Rethinking Transformer in Vision through Object Detection," arXiv, 2021.
- [14] S. Ramírez, "FastAPI," [Online]. Available: <https://fastapi.tiangolo.com>.
- [15] R. Team, "React," [Online]. Available: <https://react.dev>.
- [16] Docker, "What is Docker?," [Online]. Available: <https://docs.docker.com/get-started/docker-overview>.
- [17] S. Consortium, "About SQLite," [Online]. Available: <https://sqlite.org/about.html>.
- [18] A. S. Ariesandy, "Indonesia Traffic Sign," [Online]. Available: <https://www.kaggle.com/datasets/cakrulgaming/indonesia-traffic-sign>.